

Aula 01

Prof. Dr. Alencar Machado

Agenda

- Spring Rest
 - CRUD básico
- Documentando API Rest com SWAGGER

REST - *Re*presentational *S*tate *T*ransfer

Web Services RESTfull

O que é REST?

- Estilo arquitetural baseado em padrões WEB e no protocolo HTTP
- Roy Fielding em 2000 (CHAPTER 5)
- Arquitetura
 - Servidor REST que provê os recursos
 - Cliente REST que acessa e modifica os recursos

O que é REST?

- TUDO é um recurso em REST
- Recursos podem ter diferentes representações
 - HTML, Text, JSON, XML, ..
- Recursos são acessados por uma interface padrão através de verbos HTTP

Recurso

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/avaliador-de-projetos/alunos`. The response is displayed in the 'Body' tab, formatted as JSON. The response contains an array of three objects, with the first object expanded to show its details.

```
1  [
2    {
3      "id": 1,
4      "nome": "Alencar",
5      "matricula": "12993399",
6      "email": "alencar@ufsm.br",
7      "endereco": {
8        "complemento": "casa",
9        "bairro": "Dores",
10       "cep": "97050120",
11       "numero": "1345",
12       "cidade": "Brasilia",
13       "uf": "DF"
14     }
15   },
16   { ...
29   },
30   { ...
43   }
44 ]
```

Recurso

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/avaliador-de-projetos/aluno/2`. The response is displayed in the 'Body' tab, formatted as JSON. The response contains student information including ID, name, enrollment number, email, and address details.

GET `http://localhost:8080/avaliador-de-projetos/aluno/2`

Params Authorization Headers (6) Body Pre-request Script

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 2,
3   "nome": "Pierre",
4   "matricula": "12993390",
5   "email": "pierre@ufsm.br",
6   "endereco": {
7     "complemento": "casa",
8     "bairro": "Dores",
9     "cep": "97050120",
10    "numero": "1345",
11    "cidade": "Brasilia",
12    "uf": "DF"
13  }
14 }
```

REST: Verbos HTTP

- Verbos - CRUD
 - POST (CREATE)
 - Cria um novo recurso
 - GET (READ)
 - Define um acesso de leitura para os recurso sem ter efeitos colaterais. Recursos nunca mudam com requisições GET
 - PUT (UPDATE)
 - ATUALIZA completamente um recurso.
 - PATCH (UPDATE)
 - ATUALIZA parcialmente um recurso
 - DELETE (DELETE)
 - Remove recursos. Operação idempotente

REST: Verbos HTTP

Sl. No	Method	API end point	Description	Response status
1.	GET	/employees	Read all employees	200 (OK)
2.	POST	/employees	Create new employee	201 (CREATED)
3.	GET	/employees/{id}	Read single employee	200 (OK)
4.	PUT	/employees/{id}	Update existing employee	200 (OK)
5.	PATCH	/employees/{id}	Update existing employee	200 (OK)
6.	DELETE	/employees/{id}	Delete existing employee	200 (OK)

REST: Http status Codes

- 2xx: status de sucesso
 - 200 ok
 - 201 created
 - 204 no content
- 3xx: status de redirecionamento
 - 301 moved permanently
- 4xx: status de erro no cliente
 - 400 bad request
 - 404 not found
 - 409 conflict
- 5xx: status de erro no servidor
 - 500 internal server error

REST: Resumo

- RESTful web services são
 - Baseados em metodos HTTP e no conceito de REST
 - Definem uma URI base para os serviços
 - Suportam MIME-Types XML, text, JSON, HTML, ...,
 - Operações POST, GET, PUT, DELETE, PATCH, OPTIONS, HEAD, ..

Spring REST

- Baseado em anotações
 - @RestController
 - @RequestMapping("/aluno")
 - @PostMapping
 - @GetMapping
 - @PutMapping
 - @DeleteMapping
 - @RequestBody
 - @PathVariable
 - @PathParam
 - @Transactional

Spring REST

- Criar o AlunoService
- Criar o AlunoController
- Consumir os Endpoits

```
@Service
public class AlunoService {

    private final AlunoRepository repository;

    public AlunoService(AlunoRepository repository) { this.repository = repository; }

    public void salvar(Aluno aluno) { this.repository.save(aluno); }
    public List<Aluno> listar() {
        return this.repository.findAll();
    }
    public Aluno getAluno(Long id) {
        return this.repository.findById(id).get();
    }
    public void excluir(Long id) { this.repository.deleteById(id); }
    /*
        Para atualizar uma entidade do banco é necessário pegar a referência desta
        entidade e atualizar com os dados que vieram por parametro.
        O save(...) detecta que esse RECURSO já existe no banco de dados pela busca por id
        assim ao executar o save com id ele faz um UPDATE
    */
    public void atualizar(Aluno aluno){
        Aluno a = this.repository.getReferenceById(aluno.getId());
        a.setNome(aluno.getNome());
        a.setEmail(aluno.getEmail());
        a.setMatricula(aluno.getMatricula());
        a.setEndereco(aluno.getEndereco());
        this.repository.save(a);
    }
}
```

```
@RestController
@RequestMapping("/aluno")
public class AlunoController {

    private AlunoService service;

    public AlunoController(AlunoService service) { this.service = service; }

    /* Testar esse método primeiro
    http://localhost:8080/avaliador-de-projetos/aluno/listar */
    @GetMapping("/listar")
    public List<Aluno> listar(){
        return this.service.listar();
    }

    /* Testar esse método primeiro
    http://localhost:8080/avaliador-de-projetos/aluno/1 */
    @GetMapping("/{id}")
    public Aluno aluno(@PathVariable Long id){
        return this.service.getAluno(id);
    }
}
```

Criar AlunoController
@GetMapping

```
@RestController
```

```
@RequestMapping("/aluno")
```

```
public class AlunoController {
```

```
    private AlunoService service;
```

```
    public AlunoController(AlunoService service){
```

```
        this.service = service;
```

```
    }
```

```
    /*...*/
```

```
@GetMapping("/listar")
```

```
public List<Aluno> listar() { return this.service.listar(); }
```

```
    /*...*/
```

```
@GetMapping("/{id}")
```

```
public Aluno aluno(@PathVariable Long id){
```

```
    return this.service.getAluno(id);
```

```
}
```

```
/* Exemplo de uso para @RequestBody e POST
```

```
    http://localhost:8080/avaliador-de-projetos/aluno/print-json */
```

```
@PostMapping("/print-json")
```

```
public void printJson(@RequestBody String json){
```

```
    System.out.println(json);
```

```
}
```

```
/* http://localhost:8080/avaliador-de-projetos/aluno */
```

```
@PostMapping()
```

```
public void salvar(@RequestBody Aluno aluno){
```

```
    this.service.salvar(aluno);
```

```
}
```

```
}
```

Criar AlunoController

@PostMapping

POST http://localhost:8080/avaliador-de-projetos/aluno

JSON

Auth

Query

Headers 2

Docs

```
1 {
2     "nome": "Pierre",
3     "matricula": "189123",
4     "email": "pierre@gmail1",
5     "endereco": {
6         "complemento": "1345",
7         "bairro": "Lurdes",
8         "cep": "97797",
9         "numero": "123",
10        "cidiade": "Santa Maria",
11        "uf": "RS"
12    }
13 }
```

```

@RestController
@RequestMapping("/aluno")
public class AlunoController {
    private AlunoService service;
    public AlunoController(AlunoService service){
        this.service = service;
    }
    /* http://localhost:8080/avaliador-de-projetos/aluno/listar */
    @GetMapping("/listar")
    public List<Aluno> listar() { return this.service.listar(); }
    /* http://localhost:8080/avaliador-de-projetos/aluno/1 */
    @GetMapping("/{id}")
    public Aluno aluno(@PathVariable Long id) { return this.service.getAluno(id); }
    /* http://localhost:8080/avaliador-de-projetos/aluno/print-json */
    @PostMapping("/print-json")
    public void printJSON(@RequestBody String json) { System.out.println(json); }
    /* http://localhost:8080/avaliador-de-projetos/aluno */
    @PostMapping()
    public void salvar(@RequestBody Aluno aluno) { this.service.salvar(aluno); }

    /* http://localhost:8080/avaliador-de-projetos/aluno */
    @PutMapping
    public void atualizar(@RequestBody Aluno aluno){
        this.service.atualizar(aluno);
    }
    /* http://localhost:8080/avaliador-de-projetos/aluno/1 */
    @DeleteMapping("/{id}")
    public void deletar(@PathVariable Long id){
        this.service.excluir(id);
    }
}

```

Utilizando UUID

```
import java.util.UUID;
```

```
public interface AlunoRepository extends JpaRepository<Aluno, Long> {  
    public Aluno findAlunoByUuid(UUID uuid);  
    public void deleteAlunoByUuid(UUID uuid);  
}
```

```
@Service  
public class AlunoService {
```

```
    repository repository;
```

```
    public AlunoRepository repository() { this.repository = repository; }
```

```
    public void salvar(Aluno aluno) { this.repository.save(aluno); }
```

```
    public List<Aluno> listar() { return this.repository.findAll(); }
```

```
    public Aluno findById(Long id) { return this.repository.findById(id).get(); }
```

```
    public void deletar(Long id) { this.repository.deleteById(id); }
```

```
    public void atualizar(Aluno aluno){...}
```

```
    public void atualizarUUID(Aluno aluno){  
        Aluno a = this.repository.findAlunoByUuid(aluno.getUuid());  
        a.setNome(aluno.getNome());  
        a.setEmail(aluno.getEmail());  
        a.setMatricula(aluno.getMatricula());  
        a.setEndereco(aluno.getEndereco());  
        this.repository.save(a);  
    }
```

```
    public Aluno getAlunoUUID(String uuid) {  
        UUID uuidformatado = UUID.fromString(uuid);  
        return this.repository.findAlunoByUuid(uuidformatado);  
    }
```

```
    public void deletarUUID(String uuid){  
        this.repository.deleteAlunoByUuid(UUID.fromString(uuid));  
    }
```

```
}
```

```

@RestController
@RequestMapping("/aluno")
public class AlunoController {
    private AlunoService service;
    public AlunoController(AlunoService service){
        this.service = service;
    }
    /* http://localhost:8080/avaliador-de-projetos/aluno/listar */
    @GetMapping("/listar")
    public List<Aluno> listar() { return this.service.listar(); }
}

```

... Continuação ...

```

/*
http://localhost:8080/avaliador-de-projetos/aluno/uuid/edddd513-9ecd-45ac-8387-16256eae723a
*/
@GetMapping("/uuid/{uuid}")
public Aluno aluno(@PathVariable String uuid){
    return this.service.getAlunoUUID(uuid);
}
/*
http://localhost:8080/avaliador-de-projetos/aluno/uuid/edddd513-9ecd-45ac-8387-16256eae723a
*/
@PutMapping("/uuid")
public void atualizarUUID(@RequestBody Aluno aluno){
    this.service.atualizarUUID(aluno);
}

```

... Ocultando ID ...

PUT http://localhost:8080/avaliador-de-projetos/aluno/uuid

JSON

Auth

Query

Headers 2

Docs

```

1 {
2   "uuid": "edddd513-9ecd-45ac-8387-16256eae723a",
3   "nome": "José UUID",
4   "matricula": "189344",
5   "email": "pietra@gmail",
6   "endereco": {
7     "complemento": "1345",
8     "bairro": "Lurdes",
9     "cep": "97797",
10    "numero": "123",
11    "cidade": "Santa Maria",
12    "uf": "RS"
13  }
14 }

```

Documentando API Rest

Swagger Documentation

<https://swagger.io/docs/>